Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

Lectures

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forecast, boosting
- Lecture 14: Principal component analysis
- Lecture 15: Clustering algorithms
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

14. Bagging, Random Forests, Boosting

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2024

Bagging, random forecast, boosting

'Bagging, random forecast, boosting' are the common techniques to improve tree-based methods.

These techniques usually involve with conbining 'multiple weak learners' to order to obtain a single and very powerful model.

Bagging: sampling the training data (with replacement) to build multiple sets of training data for building multiple weak learners (classifiers/regressors). The output of the bagged model is derived these weak leaners.

Random forecast: Random forecast is built on the idea of bagging, but choose to randomly selected features to build each tree. This is to avoid building multiple similar trees, which is a tweak *to decorrelate the trees*.

Boosting: Boosting is based on a similar idea of bagging, except that the trees are grown sequentially: each tree is grown using information from previously grown trees.

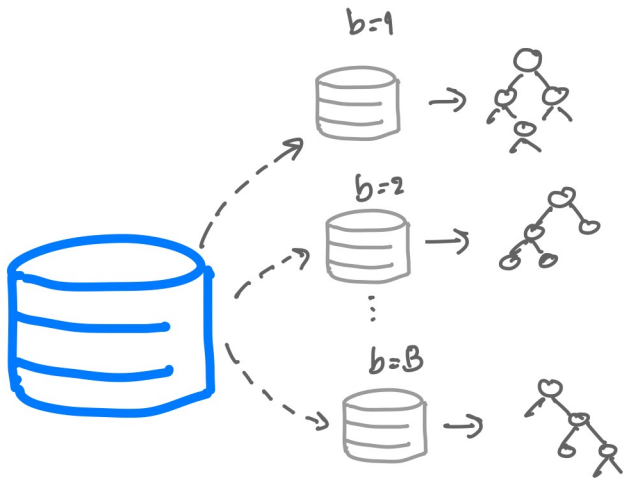
Bagging

Bagged tree:

Generate B trees using the B bootstrapped training sets (sampling training data with replacement). Then, aggregate the outputs from these B trees to be the output.

This technique can help reduce tree variance.

Boostrap, aggregation, bagging $\rightarrow$ a technique to reduce model variance.



Bagged tree

Ensemble based

Bagged regression tree:

The output is the average of the predictions from B trees:

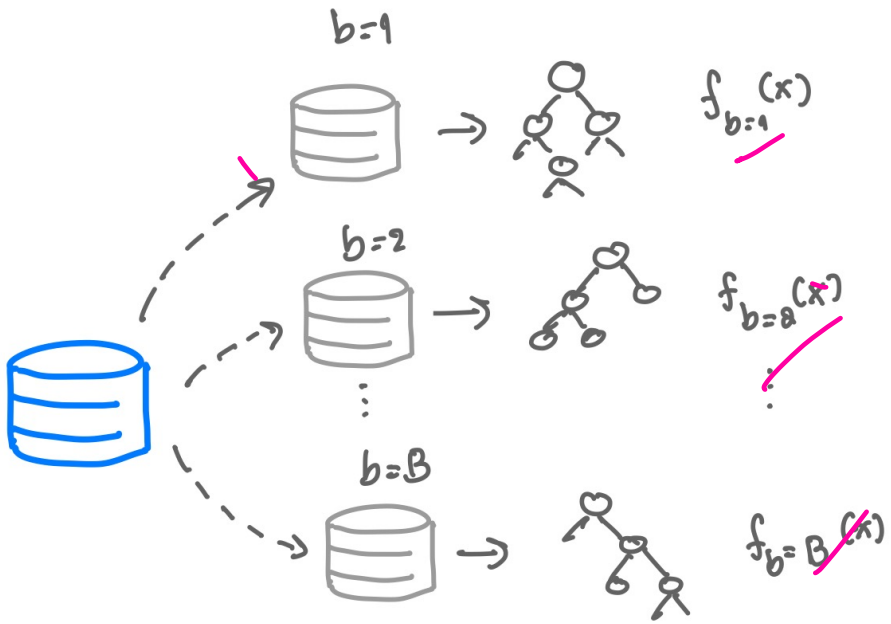
$$f_{\text{Bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$$

Weak Learner

Bagged classification tree:

The output is the majority vote of the predictions from B trees:

$$f_{\text{Bag}}(x) = \text{Majority}\{f_b(x)|b \in \{1, 2, ..., B\}\}$$



Out-of-bag error estimation

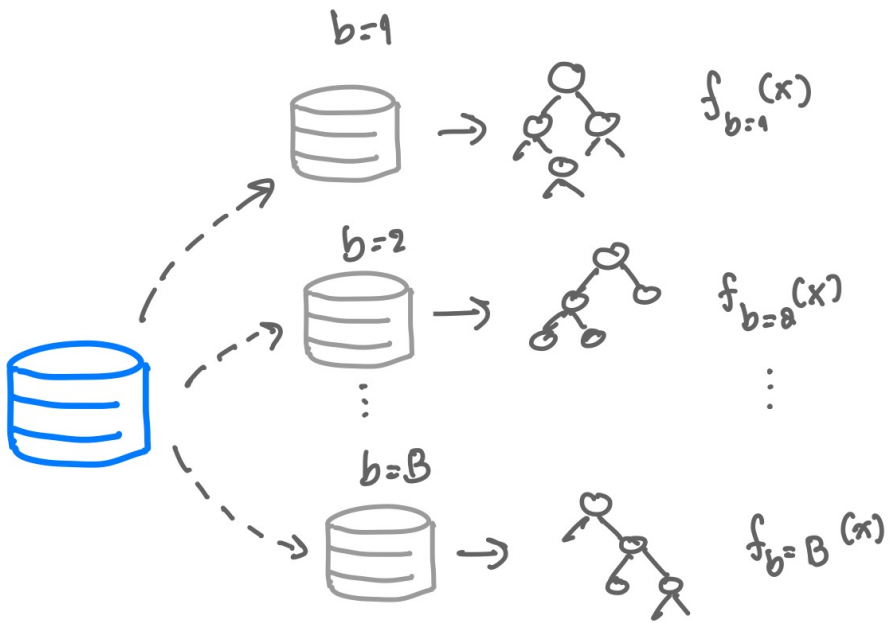
Out-of-bag error estimation

Approximately two-thirds (1/3) of the data is used for training each bagged tree.

The remaining samples are called the out-of-bag (OOB) samples.

Out-of-bag error (OOB error)

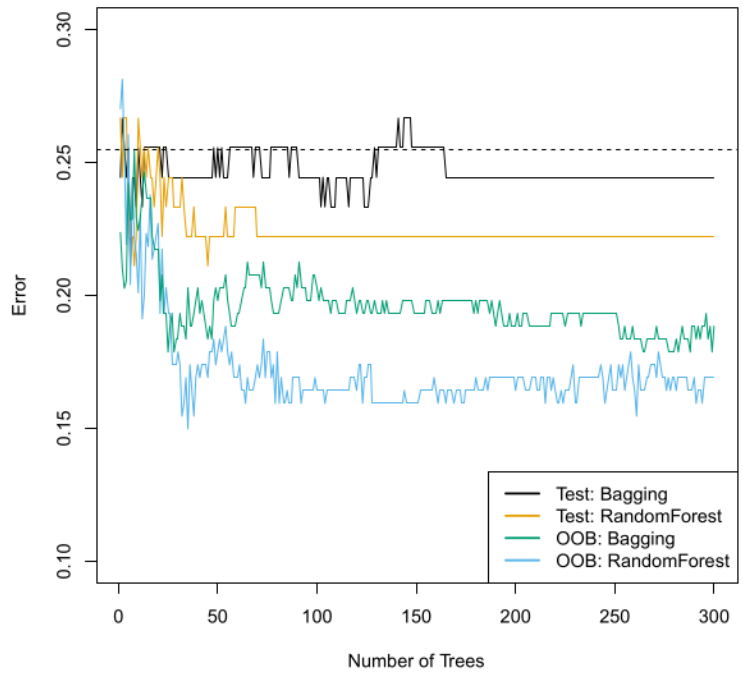
The remaining one-third (the 'out-of-bag' samples) serves as a built-in validation set.The OOB error is then calculated by averaging the predictions for each sample using only the trees where that sample was excluded from training.



Out-of-bag error performance

- **Validation Set.** You must manually "set aside" a portion of your data (e.g., 20%) before training. The model never sees this data during the entire training process.
- **OOB Error.** You use your entire dataset for the Random Forest. Because of bootstrapping (sampling with replacement), the "leftover" 36.8% of data for each tree naturally acts as a validation set. **No data is wasted**.

* If you have **a small dataset**, OOB error is your best friend because it allows you to train your trees on as much data as possible. However, if you have **a massive dataset**, a standard Validation Set is often preferred because it is computationally faster and provides a cleaner "final exam" for the model.



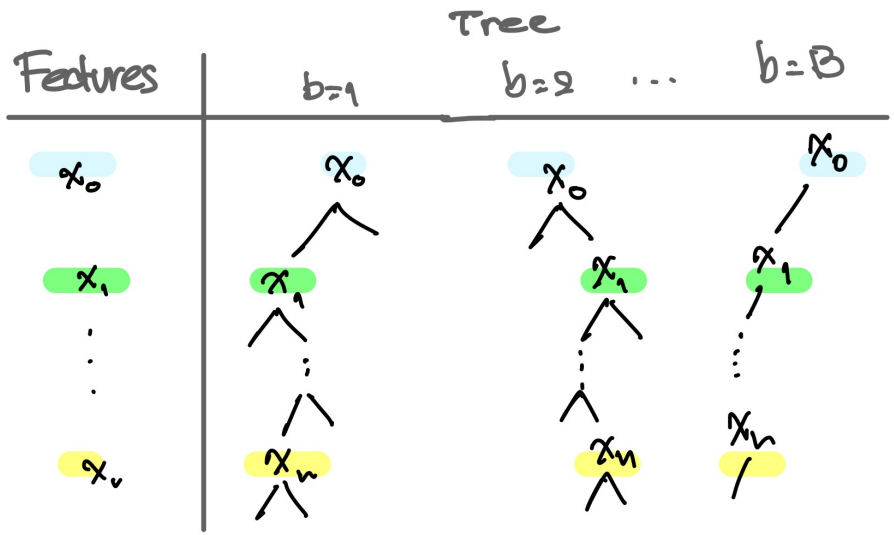
Variable importance measures

Bagging typically results in improved accuracy over prediction using a single tree. Unfortunately, using multiple trees, however, comes at a cost of losing the main advantage of the trees, *i.e.*, its interpretability.

Yet, one can obtain an overall summary of the importance of each feature using:

- variance reduction for bagging regression trees, or
- information gain (IG) based on the Gini index for bagging classification trees.

e.g., in the case of bagging regression trees, we can record the total amount that the RSS (or variance) is decreased due to splits over a given feature, averaged over all B trees. A large RSS/variance reduction indicates an important feature.

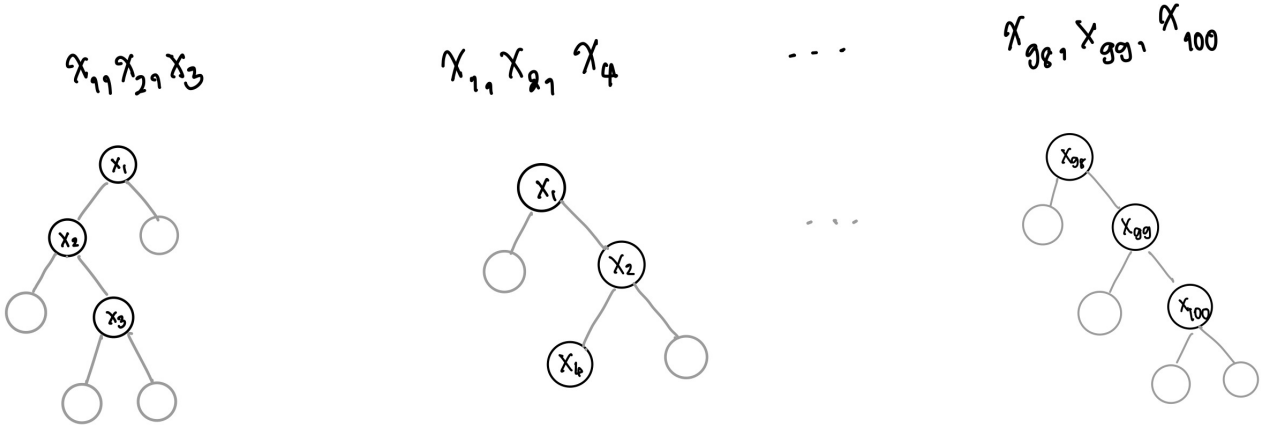


Random forest

Random forecast follows a similar idea to employ multiple trees as bagging, but it builds each tree with randomly selected features.

- Specifically, following bagging techniques, we boostrap the training dataset for building multiple trees $b = 1, ..., B$.

- Instead of using the entire set of features for each tree. Random forest will build each tree using the randomly selected p features out of n features. **This is to ensure that each tree will be different (or decorrelated)**.



- This is opposite to when the entire set of features is used ($p = n$), some of the features can be overly emphasized, causing the highly correlated tree..., which is the problem that can happen in a bagged tree.

Random forest for regression and classification

→ We will build a forest containing multiple trees: $b = 1, ..., B$ trees. The training data is bootstrapped for each tree...

For each tree, the features are randomly selected, *i.e.*, p features out of n features.

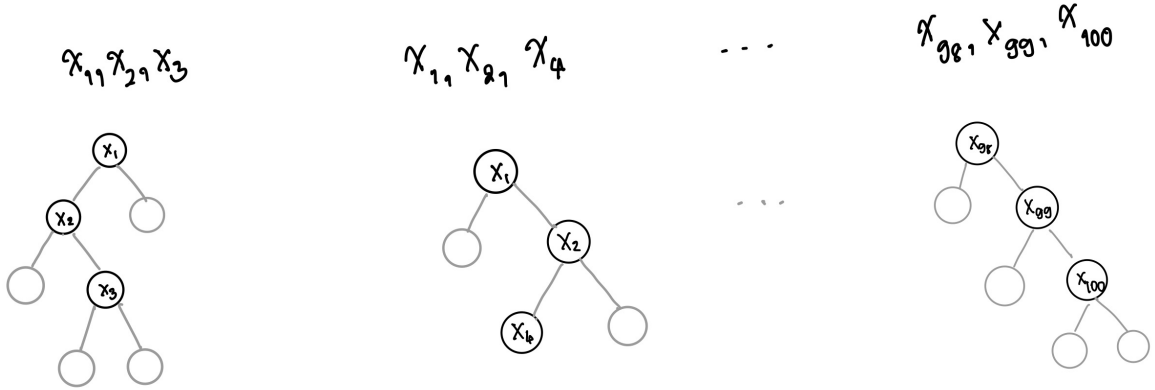
Let C_b^p be the index set of randomly selected features for the bth tree. We denote the selected features as $[x_i]_{i \in C_b^p}$ which is a subset in x

Random forest regression tree: the output is an averaged prediction from all the B trees:

$$f_{\text{Forest}}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b([x_i]_{i \in C_b^p})$$

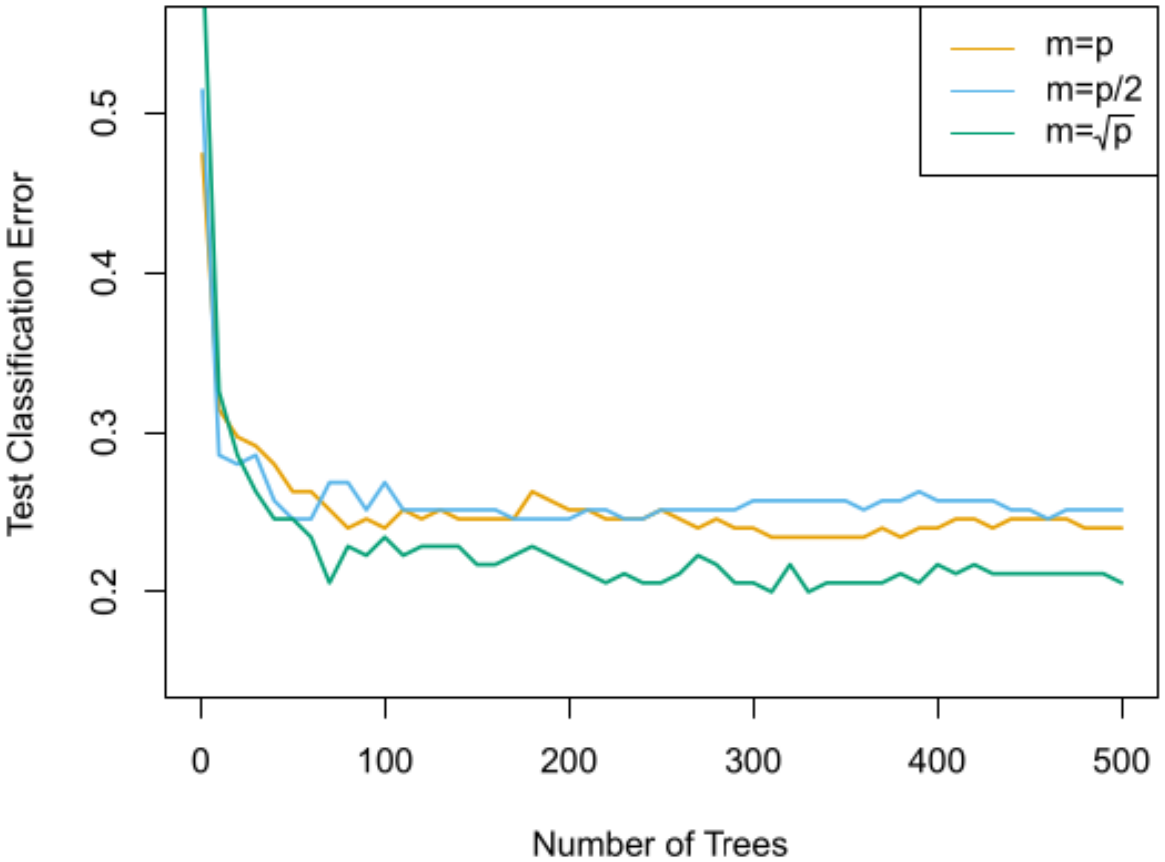
Random forest classification tree: the output is a majority vote of the B trees:

$$f_{\text{Forest}}(x) = \text{Majority}\{f_b([x_i]_{i \in C_b^p}) |\ b \in \{1, 2, ..., B\}\}$$



Random forest for classification

- Here m denotes the number of features used in building a tree. The plot below show the classification error against the increased number of trees used in bagging.
- The result below shows that, as the number of trees used in bagging increases, the lower number of features (m) leads to the better result.



5 Boosting
- Boosting concept
- Boosting vs bagging
- Boosting tree
- Gradient boosting

Boosting

Boosting concept: learn $\cup_{f_{\text{Boost}}}(x)$ as a aggregation of weak learners.

$$\cup_{f_{\text{Boost}}}(x) = \text{Aggregate}(\{\lambda f_b(x)\}_{b=1,\ldots,B})$$

where $f_b(x)$ is a weak learner, and λ is a scaling factor.

Then, the aggregated tree will be used to provide the prediction in the following manner ...

Regression: The prediction is the sum prediction from B trees:

$$f_{\text{Boost}}(x) = \sum_{b=1}^{B} \beta f_b(x) \qquad \text{where} \qquad f_b(x) \in \cup_{f_{\text{Boost}}}(x)$$

and β is an appropriate parameter for computing the overall prediction.

Classification: The prediction is the majority vote of the B trees:

$$f_{\text{Boost}}(x) = \text{Majority}(\{f_b(x) \in \cup_{f_{\text{Boost}}}(x)\})$$

Boosting vs bagging

What makes boosting different from bagging?

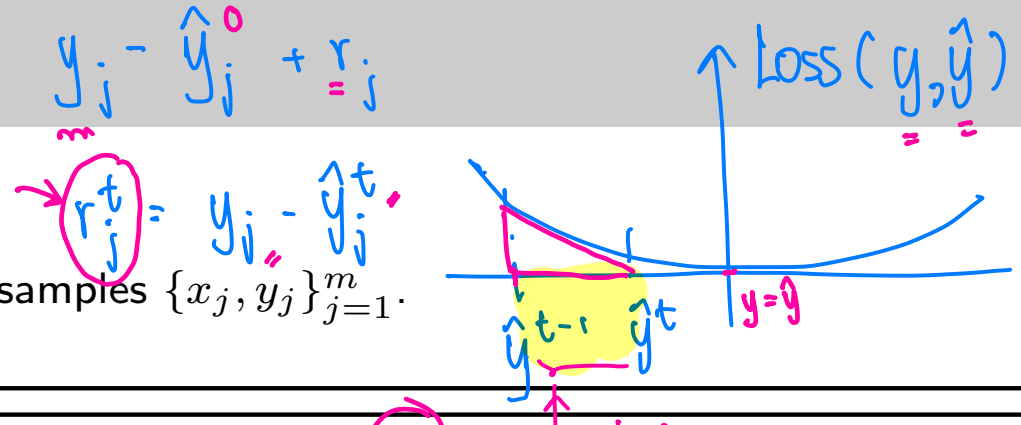
- The idea is to learn the weak learner $f^t(x)$ can complete the result of the boosted tree $f^t_{\text{Boost}}(x)$.
- The weak learner in boosting are learned **sequentially**, which is different from bagging and random forest whose weak learners can be learned in parallel as they are totally independent of one another.
- As a result, the weak leaner $f_b(x)$ is no longer just a tree that is trained with bootstrapped training data, but it is trained to boost the performance of **the previous version** of the aggregated trees.

So, please keep these two points in mind, we will now go though how to build a tree using boosting technique ?

Let us define the following notations

- Aggregate$(\cdot)$ denotes an aggregation opertation such as summation or union.
- Train$(\cdot)$ denotes a training proceedure whose output is a trained model.

Boosting tree algorithm



Input Let $\mathcal{T}$ denote the set of all training samples $\{x_j, y_j\}_{j=1}^m$.
Shrinkage parameter λ

1: Initialize the prediction $\hat{y}^t = \bar{y}$.
2: **for** $t = 1$ to T **do**
3: Compute the residual for $j = 1, ..., m$ and train a weak learner to predict the given residual:

$$r_j^t = y_j - \hat{y}_j^{t-1} \qquad f^t(\cdot; \beta_t) = \text{Train}(\{(x_j, r_j^t)\}_{(x_j, r^t(x_j)) \in \mathcal{T}}$$

4: Aggregate the tree (for all $x_j \in \mathcal{T}$)

$$f_{\text{Boost}}^t(x_j; \lambda) := \text{Aggregate}(\{f_{\text{Boost}}^{t-1}(x_j), \lambda f^t(x_j; \beta_t)\})$$

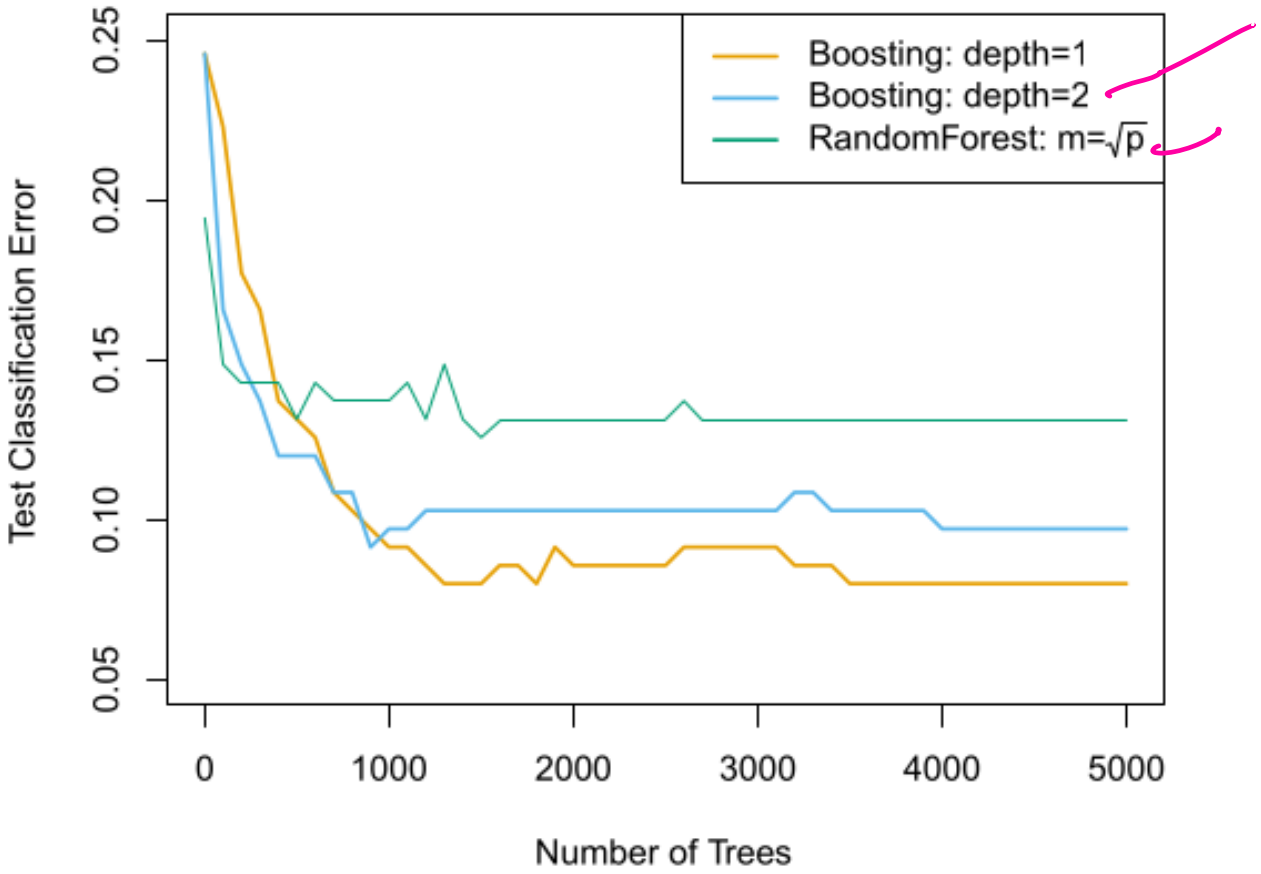
5: Update prediction (for all $x_j \in \mathcal{T}$):

$$\hat{y}_j^t = f_{\text{Boost}}^t(x_j; \lambda)$$

6: **end for**
7: **Output** $f_{\text{Boost}}(\cdot) = \text{Aggregate}(\lambda\{f_t(\cdot; \beta_t)\}_{t=1}^T)$

Boosting tree algorithm

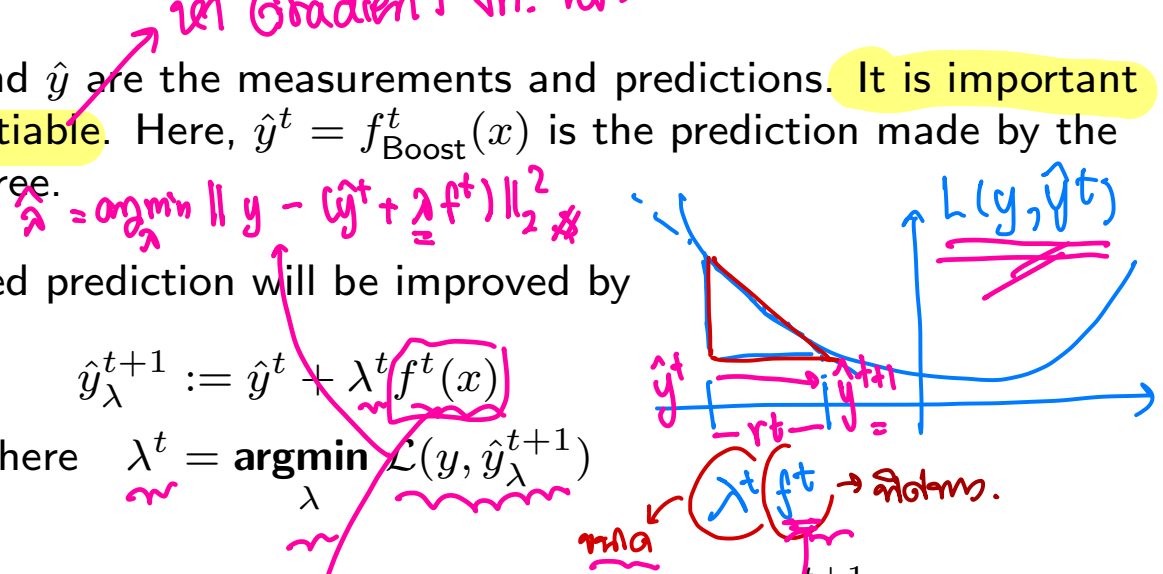
- The result below shows that the increased depth in a boosting tree offers a similar result to a random forest with number of features $\sqrt{p}$.



Gradient boosting's idea

The idea is to optimize λ such that the weak learner $f^t(x)$ can complete the result of the boosted tree $f^t_{\text{Boost}}(x)$.

Define a loss $\mathcal{L}(y, \hat{y})$ where y and $\hat{y}$ are the measurements and predictions. It is important that the loss function is differentiable. Here, $\hat{y}^t = f^t_{\text{Boost}}(x)$ is the prediction made by the current version of the boosted tree.



- In gradient boosting, the updated prediction will be improved by

$$\hat{y}^{t+1}_\lambda := \hat{y}^t + \lambda^t f^t(x)$$

$$\text{where} \quad \lambda^t = \underset{\lambda}{\textbf{argmin}}\, \mathcal{L}(y, \hat{y}^{t+1}_\lambda)$$

- As a result, the weak learner should be updated in a direction to which $\hat{y}^{t+1}_\lambda$ moves closer to the measurement y. The weak learner is then updated by:

$$f^t(x) \approx \lim_{\Delta \to 0} \frac{\mathcal{L}(y, \hat{y}^t + \Delta) - \mathcal{L}(y, \hat{y}^t)}{\hat{y}^t + \Delta - \hat{y}^t}$$

$$\approx -\frac{d}{d\hat{y}^t} \mathcal{L}(y, \hat{y}^t + \lambda f^t(x))$$

- The gradient boosting algorithm will optimize the model by updating λ and $f^t(x)$ alternately.

Gradient boosting algorithm

Input The set of training samples $\mathcal{T} = \{x_j, y_j\}_{j=1}^m$
Define a loss function $\mathcal{L}(y, \hat{y})$ where y and $\hat{y}$ are the measurements and predictions.

1: Initialize $\hat{y}_j^{t=0}$ for $j = 1, 2, ...m$. Compute the residual for $j = 1, 2, ...m$ and update weak learner using the given residual:

$$r_j^t = y_j - \hat{y}_j^{t-1} \qquad f_t(\cdot; \beta_t) = \text{Train}(\{x_j, r_j^t\}_{j=1}^m)$$

2: **for** $t = 1$ to T **do**
3: Optimize the shrinkage parameter.

$$\lambda_t = \underset{\lambda}{\textbf{argmin}} \sum_{j=1}^{m} \mathcal{L}(y_j, \hat{y}^{t-1} + \lambda f^t(x_j; \beta_t))$$

4: Update boosted tree $f_{\text{Boost}}(\cdot) = \text{Aggregate}(\{f_{\text{Boost}}(\cdot), \lambda_t f_t(\cdot; \beta_t)\})$. Then, update $\hat{y}^t$. For all $x_j \in \mathcal{T}$,

$$\hat{y}_j^t = f_{\text{Boost}}(x_j)$$

5: Compute the weak learner. For all $x_j \in \mathcal{T}$,

$$f^t(x_j) = - \left. \frac{\partial \mathcal{L}(y_j, \hat{y}_j)}{\partial \hat{y}_j} \right|_{\hat{y}_j^t = f_{\text{Boost}}(x_j)}$$

6: **end for**
7: **Output** $f_{\text{Boost}}(\cdot)$

Summary

Important points and remaining challenges in each methods:

Bagging: the trees are grown independently on random samples of the observations. Consequently, the trees tend to be quite similar to each other. Thus, bagging can get caught in local optima and can fail to thoroughly explore the model space.

Random forecast: the trees are once again grown independently on random samples of the observations. However, each split on each tree is performed using a random subset of the features, thereby decorrelating the trees, and leading to a more thorough exploration of model space relative to bagging.

Boosting: we only use the original data, and do not draw any random samples. The trees are grown successively, using a *slow* learning approach: each new tree is fit to the signal that is left over from the earlier trees, and shrunken down before it is used.

References I

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html.

[3] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 9. Support vector machine. URL: http://jitkomut.eng.chula.ac.th/ee575.html.